PART II: RELATIONAL MODEL, ALGEBRA & SQL • Chapter 5

The Relational Model, Keys, and Integrity Constraints

CHAPTER 5 ROADMAP & LEARNING OBJECTIVES

Why This Matters: *Why does a relational table reject duplicate Primary Keys, forbid orphan Foreign Keys, and behave counterintuitively when NULL values appear in WHERE or NOT IN clauses? This chapter builds the mathematical foundation of E.F. Codd's Relational Model—covering Degree vs. Cardinality, all 8 Key types, GATE CS combinatorial formulas for counting Super Keys, Referential Triggered Actions, and Kleene's Three-Valued Logic (3VL).*

Prerequisites: Chapters 1–4 (DBMS Fundamentals, Schemas vs. Instances, and ER-to-Relational Mapping).

Learning Objectives (What You Will Master in This Chapter):

- Define Relation Schema, Relation Instance, Tuple, Attribute, Domain, Degree (Arity), and Cardinality mathematically, and contrast Set Semantics with SQL Bag (Multiset) Semantics.
- Master all 8 Key types (Super Key, Candidate Key, Primary Key, Alternate/Secondary Key, Composite Key, Prime vs. Non-Prime Attributes, Foreign Key, and Surrogate vs. Natural Key).
- Derive and apply the exact GATE CS combinatorial formulas (including Inclusion–Exclusion and Sperner's Theorem) to count Super Keys for any Candidate Key configuration.
- Enforce Domain Constraints, Key Constraints, Entity Integrity (PK never NULL), and Referential Integrity (FK matches PK or is NULL), and debunk the 5 classic Foreign Key myths.
- Trace DML operations (INSERT, DELETE, UPDATE) across Parent and Child tables under all 5 SQL Referential Actions (RESTRICT, NO ACTION, CASCADE, SET NULL, SET DEFAULT).
- Evaluate complex predicates under SQL Three-Valued Logic (TRUE, FALSE, UNKNOWN) and distinguish how WHERE (passes TRUE only) differs from CHECK constraints (passes TRUE or UNKNOWN).

Contents

5 The Relational Model, Keys, and Integrity Constraints **1**

5.1 Mathematical Foundation: Relation, Tuple, Attribute, Domain, Degree & Cardinality **3**

5.1.1 Formal Mathematical Definitions & Visual Anatomy of a Relation **3**

5.1.2 Three-Way Vocabulary Translation & Four Properties of Mathematical Relations **4**

5.2 Complete Taxonomy of Keys in the Relational Model **5**

5.2.1 Formal Definitions of All Eight Key Types & Set Inclusion Hierarchy **5**

5.2.2 GATE CS Masterclass: Combinatorial Formulas for Counting Super Keys **7**

5.2.3 Worked Example 5.1: Solving Four GATE Super-Key Counting Problems **8**

5.3 Relational Integrity Constraints (Domain, Entity & Referential Integrity) **9**

5.3.1 Domain, Key, and Entity Integrity Constraints **9**

5.3.2 Referential Integrity (Foreign Key Rules) & 5 Classic Foreign Key Myths **10**

5.4 Referential Triggered Actions (ON DELETE and ON UPDATE) **11**

5.4.1 Table 5.2: DML Violation Matrix & The 5 Standard SQL Referential Actions **11**

5.4.2 Worked Examples 5.2 & 5.3: Multi-Level Cascade Trace & Complete UniVerseDB DDL **12**

5.5 NULL Semantics and Three-Valued Logic (3VL: TRUE, FALSE, UNKNOWN) **14**

5.5.1 Complete Truth Tables for SQL 3VL (AND = min, OR = max, NOT = 1 − P) **14**

5.5.2 Worked Example 5.4: Tracing 3VL Queries & The WHERE vs. CHECK Asymmetry **15**

5.6 Chapter 5 Summary & One-Page Rapid-Revision Cheat Sheet **16**

5.7 Chapter 5 Exam, GATE & Placement Question Bank (Parts A–H) **18**

5.8 Chapter 5 Complete Answer Key & Worked Solutions **22**

5.1 Mathematical Foundation: Relation, Tuple, Attribute, Domain, Degree & Cardinality

In 1970, E.F. Codd grounded database management in set theory and first-order predicate logic. What programmers casually call a "table" is formally a mathematical Relation—a subset of the Cartesian product of attribute domains.

5.1.1 Formal Mathematical Definitions

1. Domain (D_i): A pool of atomic (indivisible) values of a specific data type and semantic meaning from which attribute values are drawn (e.g., D_CGPA = real numbers in [0.00, 10.00] with 2 decimal places).
2. Relation Schema R(A_1, A_2, ..., A_n): The structural intension consisting of a relation name R and a list of attributes A_1, ..., A_n, where each A_i is the name of a role played by domain dom(A_i).
3. Degree (or Arity, n): The total number of attributes (columns) in the relation schema R. A relation of degree 1 is unary, degree 2 is binary, degree 3 is ternary, and degree n is n-ary.
4. Tuple (t): An ordered list of n values t = <v_1, v_2, ..., v_n>, where each v_i ∈ dom(A_i) or is the special NULL marker.
5. Relation Instance r(R): A finite set of m tuples r(R) = {t_1, t_2, ..., t_m}, which is a subset of the Cartesian product: r(R) ⊆ dom(A_1) × dom(A_2) × ... × dom(A_n).
6. Cardinality (m or |r|): The total number of tuples (rows) present in the relation instance r(R) at a given moment.

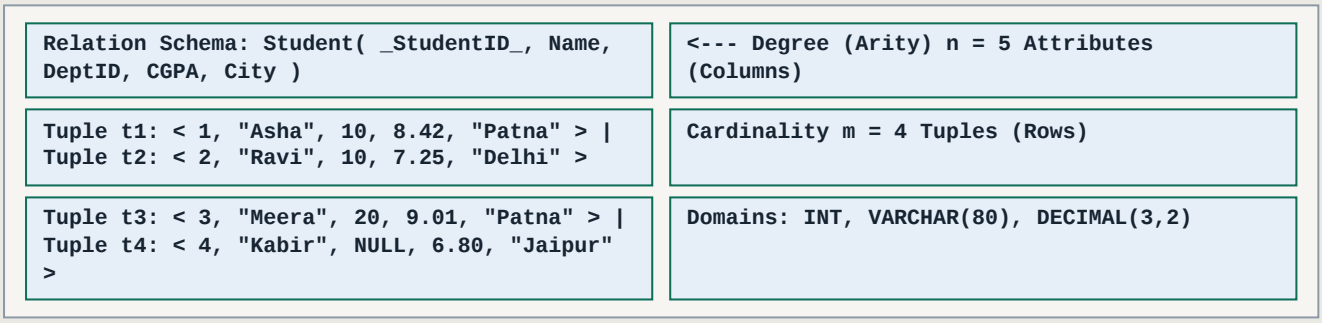


Figure 5.1: Visual Anatomy of the Student Relation showing Degree (Arity n = 5 columns) and Cardinality (m = 4 tuples).

Table 5.1: Three-Way Vocabulary Translation Across Relational Theory, SQL, and File Systems

Formal Relational Model Term	SQL / Commercial RDBMS Term	Traditional File-System Term
Relation (or Relation Instance)	Table	File / Dataset
Tuple (n-tuple)	Row	Record
Attribute	Column	Field
Domain	Data Type + CHECK Constraint	Field Format / Value Pool
Degree (Arity, n)	Number of Columns	Number of Fields per Record
Cardinality (m)	Number of Rows	Number of Records in File

5.1.2 Four Mathematical Characteristics of Relations (Set vs. Bag Semantics)

1. 1. No Duplicate Tuples (Set Semantics): Because a formal relation r(R) is a mathematical SET of tuples, duplicate rows cannot exist in a pure relation. (Note: Commercial SQL tables without a Primary Key/UNIQUE constraint use Bag/Multiset semantics and allow duplicate rows unless SELECT DISTINCT is used).
2. 2. Ordering of Tuples Is Immaterial: A set {t1, t2, t3} is identical to {t3, t1, t2}. Rows in a relation have no inherent positional order unless sorted explicitly via ORDER BY.
3. 3. Ordering of Attributes Is Logically Immaterial: When tuples are defined as functions mapping attribute names to values (t[A_i] = v_i), column order does not affect relational semantics.
4. 4. First Normal Form (1NF — Atomicity of Values): Every value v_i in every tuple must be atomic (indivisible). Composite values, nested tables, and multi-valued sets inside a single cell are forbidden in the pure Relational Model.

5.2 Complete Taxonomy of Keys in the Relational Model

Because duplicate tuples are forbidden in a mathematical relation, we need a formal mechanism to uniquely identify every tuple across all possible valid instances of a schema:

5.2.1 Formal Definitions of All Eight Key Types

1. Super Key (SK): Any attribute or set of attributes SK ⊆ R that uniquely identifies every tuple in any valid instance of R: for all distinct tuples t1 ≠ t2, t1[SK] ≠ t2[SK]. Note: The entire attribute set R is always a (trivial) Super Key; adding ANY extra attribute to a Super Key still yields a Super Key!
2. Candidate Key (CK — Minimal Super Key): A Super Key from which NO proper subset is also a Super Key (Uniqueness + Irreducibility/Minimality). A relation can have multiple Candidate Keys (e.g., {StudentID}, {AadhaarNo}, {RollNo}).
3. Primary Key (PK): The single Candidate Key chosen by the database designer as the principal identifier of tuples in R. In SQL, the Primary Key is additionally constrained to be NOT NULL.
4. Alternate Key (Secondary Key): Any Candidate Key that was NOT chosen as the Primary Key: Alternate Keys = Candidate Keys − {Primary Key}. Implemented in SQL using UNIQUE.
5. Composite Key (Compound Key): Any key (Candidate, Primary, or Foreign) consisting of two or more attributes (e.g., (StudentID, SectionID) in Enrollment).
6. Prime vs. Non-Prime Attribute: An attribute A is a Prime Attribute if A is a member of AT LEAST ONE Candidate Key of R (even an Alternate Key!). Otherwise, A is a Non-Prime Attribute.
7. Foreign Key (FK): A set of attributes in a referencing (child) relation R1 that references the Primary Key (or UNIQUE key) of a referenced (parent) relation R2.
8. Surrogate Key vs. Natural (Business) Key: A Natural Key is a real-world attribute (e.g., ISBN, AadhaarNo, PAN). A Surrogate Key is a system-generated artificial identifier (e.g., SERIAL, IDENTITY, UUID) with no business meaning.

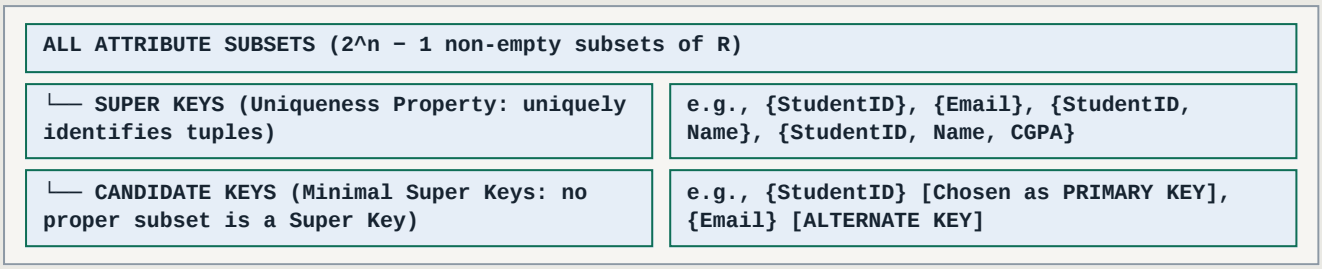


Figure 5.2: Set Inclusion Hierarchy of Keys — Every Primary Key is a Candidate Key, and Every Candidate Key is a Super Key.

5.2.2 GATE CS Masterclass: Combinatorial Formulas for Counting Super Keys

Given a relation R(A_1, A_2, ..., A_n) with n attributes, every Superset of any Candidate Key is a Super Key! Use these exact combinatorial formulas in GATE CS:

Exact Combinatorial Formulas for Counting the Number of Super Keys of R(A_1, ..., A_n)

Candidate Key Configuration on n Attributes	Exact Formula for Total Number of Super Keys	Derivation / Explanation
Case 1: Exactly 1 Simple Candidate Key {A_1} (size k = 1)	2^(n − 1)	A_1 must be present (1 choice); each of the remaining (n − 1) attributes can be included or excluded (2 choices each).
Case 2: Exactly 1 Composite Candidate Key of size k	2^(n − k)	All k attributes of the CK must be present; the remaining (n − k) attributes have 2^(n−k) combinations.
Case 3: Two Disjoint Candidate Keys K1 (size p) and K2 (size q) with K1 ∩ K2 = ∅	2^(n − p) + 2^(n − q) − 2^(n − (p + q))	By Principle of Inclusion–Exclusion: |S(K1) ∪ S(K2)| = |S(K1)| + |S(K2)| − |S(K1 ∪ K2)|.
Case 4: Two Overlapping Candidate Keys K1, K2 with |K1| = p, |K2| = q, and |K1 ∪ K2| = u	2^(n − p) + 2^(n − q) − 2^(n − u)	In the intersection term, any superset of BOTH K1 and K2 must contain all u = |K1 ∪ K2| distinct attributes!
Case 5: m Simple (1-Attribute) Candidate Keys {A_1}, {A_2}, ..., {A_m}	2^n − 2^(n − m) = 2^(n − m) × (2^m − 1)	Complement Counting: Total subsets (2^n) minus subsets containing NONE of the m key attributes (2^(n−m)).
Max Possible Candidate Keys on n Attributes (Sperner's Theorem)	C(n, floor(n / 2))	Because no CK can be a subset of another CK (antichain), max CKs occurs when all CKs have size floor(n/2).

5.2.3 Worked Example 5.1: Solving Four GATE Super-Key Counting Problems

WORKED EXAMPLE 5.1 | Four Classic GATE CS Super-Key Counting Problems

Problem 1 (Disjoint Composite CKs): R(A, B, C, D, E, F) has n = 6 attributes and two Candidate Keys: {A, B} (size 2) and {C, D} (size 2).
• Solution: |SK| = 2^(6−2) + 2^(6−2) − 2^(6−4) = 2^4 + 2^4 − 2^2 = 16 + 16 − 4 = 28 Super Keys.

Problem 2 (Overlapping Composite CKs): R(A, B, C, D, E, F) has n = 6 attributes and two Candidate Keys: {A, B} and {B, C}.
• Solution: Here p = 2, q = 2, and K1 ∪ K2 = {A, B, C} has size u = 3.
|SK| = 2^(6−2) + 2^(6−2) − 2^(6−3) = 16 + 16 − 8 = 24 Super Keys.

Problem 3 (Three Simple CKs — Complement Method): R(A, B, C, D, E) has n = 5 attributes and three simple Candidate Keys: {A}, {B}, and {C} (m = 3).
• Solution: A subset is a Super Key iff it contains at least one of {A, B, C}. Number of subsets containing NONE of {A, B, C} is 2^(5−3) = 2^2 = 4.
|SK| = 2^5 − 2^2 = 32 − 4 = 28 Super Keys.

Problem 4 (Sperner's Theorem): What is the maximum number of Candidate Keys possible for a relation with n = 6 attributes?
• Solution: By Sperner's Theorem, max CKs = C(6, floor(6/2)) = C(6, 3) = (6 × 5 × 4) / (3 × 2 × 1) = 20 Candidate Keys!

COMMON MISTAKE | Inferring Candidate Keys from a Single Snapshot of 4 Rows!

Looking at a 4-row table where all 4 students happen to have different names ("Asha", "Ravi", "Meera", "Kabir") does NOT prove Name is a Candidate Key! You can only prove from an instance that an attribute is NOT a Super Key (if two rows share the same City = "Patna"); proving it IS a key requires schema-level Functional Dependencies.

5.3 Relational Integrity Constraints

Relational Integrity Constraints are formal conditions specified on a database schema that restrict the data values that can be stored in any valid instance of the database:

5.3.1 Domain Constraints, Key Constraints, and Entity Integrity Rule

1. Domain Constraint: Every attribute value t[A_i] in every tuple must be an atomic value from dom(A_i) (enforced via SQL data types, NOT NULL, and CHECK predicates such as CHECK (CGPA BETWEEN 0.00 AND 10.00)).
2. Key Constraint (Uniqueness): No two distinct tuples in any valid relation instance r(R) can have identical values across all attributes of a Candidate Key.
3. Entity Integrity Constraint: No attribute participating in the PRIMARY KEY of a base relation is ever allowed to be NULL! Even if a Primary Key is composite—such as (StudentID, SectionID) in Enrollment—NEITHER StudentID NOR SectionID can be NULL. Why? Because if part of a Primary Key were NULL (unknown), we could not uniquely identify the entity or distinguish two tuples!

5.3.2 Referential Integrity Constraint (Foreign Key Rules)

Specified between two relations—the Referencing (Child) relation R1 and the Referenced (Parent) relation R2—Referential Integrity states that for every tuple t1 in R1, the Foreign Key value t1[FK] must satisfy one of two conditions:

- Condition 1: Either t1[FK] matches the Primary Key value t2[PK] of some existing tuple t2 in parent relation R2, OR
- Condition 2: t1[FK] is wholly NULL (provided the Foreign Key column is not declared NOT NULL and is not part of R1's Primary Key).

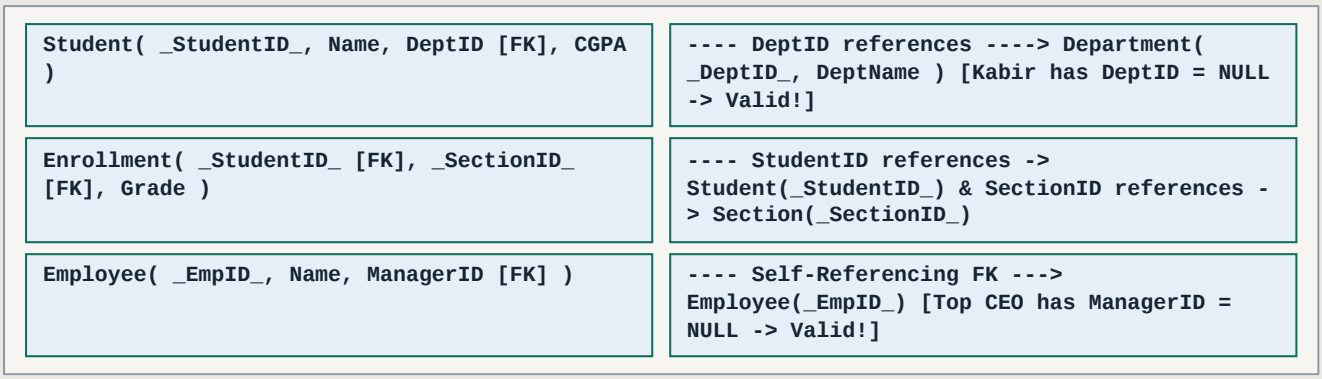


Figure 5.3: Referential Integrity Arrows in UniVerseDB (Child Referencing Tables -> Parent Referenced Tables).

EXAM MASTERCLASS | Debunking the 5 Classic Foreign Key Myths in GATE & Vivas

Myth 1: "A Foreign Key cannot be NULL." -> FALSE! A Foreign Key CAN be NULL unless explicitly constrained with NOT NULL (e.g., Student Kabir with undeclared department has DeptID = NULL).
Myth 2: "A Foreign Key cannot have duplicate values." -> FALSE! In a 1:N relationship, 400 students in CSE all have DeptID = 10 in the Student table!
Myth 3: "A Foreign Key must have the same column name as the referenced Primary Key." -> FALSE! Only the underlying Domain (data type) must match (e.g., ManagerID references EmpID).
Myth 4: "A Foreign Key must reference a different table." -> FALSE! A Foreign Key can reference the Primary Key of the SAME table (Self-Referencing Foreign Key).
Myth 5: "An Alternate Key (UNIQUE) cannot be NULL." -> FALSE! Unlike Primary Keys, SQL UNIQUE columns allow NULL values (and in standard PostgreSQL/SQL, allow multiple NULLs!).

5.4 Referential Triggered Actions (ON DELETE and ON UPDATE)

Which DML operations on a Parent table (Department) or Child table (Student) can violate Referential Integrity, and how does the DBMS respond?

Table 5.2: Complete DML Referential Integrity Violation Matrix (Parent Department vs. Child Student)

DML Operation	On Parent Table (Department)	On Child Table (Student)
INSERT	SAFE (Never violates Referential Integrity — a new parent simply has 0 children initially).	MAY VIOLATE! If inserted Student has DeptID = 99 and 99 does not exist in Department -> REJECTED!
DELETE	MAY VIOLATE! Deleting DeptID = 10 while Students still reference DeptID = 10 would leave orphans -> Triggers ON DELETE action!	SAFE (Deleting a child row never leaves orphans in the parent table).
UPDATE (PK / FK)	MAY VIOLATE! Changing DeptID = 10 to 15 while Students reference 10 -> Triggers ON UPDATE action!	MAY VIOLATE! Updating Student.DeptID to 99 (non-existent in Department) -> REJECTED!

5.4.1 The Five Standard SQL Referential Actions (ON DELETE / ON UPDATE)

1. RESTRICT (or NO ACTION — Default in SQL): Rejects the parent DELETE/UPDATE if any matching child rows exist. (RESTRICT checks immediately; NO ACTION checks at the end of the statement/constraint deferral boundary).
2. CASCADE: Automatically propagates the parent change to all matching child rows (e.g., ON DELETE CASCADE deletes all matching child rows; ON UPDATE CASCADE updates the child Foreign Key to the new parent PK).
3. SET NULL: Automatically sets the Foreign Key column in all matching child rows to NULL (valid only if the child FK column is nullable!).
4. SET DEFAULT: Sets the Foreign Key column in matching child rows to its declared DEFAULT value (which must exist in the parent table).

5.4.2 Worked Example 5.2: Multi-Level Cascading Delete & Self-Referencing Trace

WORKED EXAMPLE 5.2 | Tracing Multi-Table Referential Actions (GATE Favorite!)

Suppose we have three tables:
• Department(DeptID PK) with rows {10, 20}.
• Student(StudentID PK, DeptID FK REFERENCES Department ON DELETE SET NULL) with rows: (1, 10), (2, 10), (3, 20).
• Enrollment(StudentID FK REFERENCES Student ON DELETE CASCADE, CourseID, PK(StudentID, CourseID)) with rows: (1, "CS301"), (2, "CS301"), (3, "EC204").

Question A: What happens when we execute DELETE FROM Department WHERE DeptID = 10;?
• Trace: Department 10 is deleted. Because Student.DeptID has ON DELETE SET NULL, Students 1 and 2 are NOT deleted—their DeptID simply becomes NULL! Because no Student row was deleted, Enrollment is completely unaffected (all 3 enrollment rows remain!).

Question B: What if Student.DeptID instead had ON DELETE CASCADE and we executed DELETE FROM Department WHERE DeptID = 10;?
• Trace: Deleting Department 10 cascades and deletes Students 1 and 2 from Student; deleting Students 1 and 2 in turn triggers Enrollment's ON DELETE CASCADE, deleting (1, "CS301") and (2, "CS301")! Only Student 3 and (3, "EC204") survive!

`DELETE Department(10) + ON DELETE RESTRICT`	`---> Transaction ABORTED with Foreign Key Violation Error (All rows unchanged)`
`DELETE Department(10) + ON DELETE CASCADE`	`---> Deletes Department(10) AND automatically deletes Student(1) and Student(2)`
`DELETE Department(10) + ON DELETE SET NULL`	`---> Deletes Department(10) AND updates Student(1).DeptID = NULL, Student(2).DeptID = NULL`

Figure 5.4: Visual Comparison of RESTRICT (Aborts), CASCADE (Deletes Children), and SET NULL (Unlinks Children) on Parent Delete.

5.4.3 Worked Example 5.3: Complete SQL DDL for UniVerseDB with All Constraint Types

Listing 5.1: Production SQL DDL Demonstrating Domain, Primary Key, Alternate Key, and Referential Actions

```
CREATE TABLE Department (
    DeptID   INT PRIMARY KEY,
    DeptName VARCHAR(60) NOT NULL UNIQUE,               -- Alternate Key
    Building VARCHAR(60) NOT NULL,
    Budget   NUMERIC(12,2) NOT NULL CHECK (Budget > 0)  -- Domain Constraint
);

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,                          -- Entity Integrity (PK)
    RollNo    CHAR(9) NOT NULL UNIQUE,                  -- Alternate Key
    Name      VARCHAR(80) NOT NULL,
    DeptID    INT REFERENCES Department(DeptID)
              ON DELETE SET NULL                        -- Keep student if dept closes
              ON UPDATE CASCADE,                        -- Cascade DeptID renumbering
    CGPA      NUMERIC(3,2) CHECK (CGPA BETWEEN 0.00 AND 10.00),
    City      VARCHAR(50) DEFAULT 'Patna'
);

CREATE TABLE Enrollment (
    StudentID INT REFERENCES Student(StudentID) ON DELETE CASCADE,
    CourseID  VARCHAR(10) NOT NULL,
    Semester  VARCHAR(12) NOT NULL,
    Grade     CHAR(2) CHECK (Grade IN ('A+','A','B','C','D','F','I')),
    PRIMARY KEY (StudentID, CourseID, Semester)         -- Composite Primary Key
);
```

5.5 NULL Semantics and Three-Valued Logic (3VL: TRUE, FALSE, UNKNOWN)

In the Relational Model, NULL is NOT zero, NOT an empty string "", and NOT a space—it is a special marker indicating that a value is Unknown, Missing, or Inapplicable. Because comparing anything with an unknown value (=, <>, <, >) yields neither TRUE nor FALSE, SQL uses Kleene's Three-Valued Logic (3VL) with truth values TRUE (1), FALSE (0), and UNKNOWN (0.5):

5.5.1 Complete Truth Tables for SQL 3VL (Memory Trick: T = 1, U = 0.5, F = 0)

You never need to memorize 27 truth-table cells! Just assign FALSE = 0, UNKNOWN = 0.5, TRUE = 1 and apply three simple formulas:

• P AND Q = min(P, Q)

• P OR Q = max(P, Q)

• NOT P = 1 − P (so NOT UNKNOWN = 1 − 0.5 = 0.5 = UNKNOWN!)

Table 5.3: Master Truth Table for SQL Three-Valued Logic (3VL)

P	Q	P AND Q [min(P, Q)]	P OR Q [max(P, Q)]	NOT P [1 − P]
TRUE (1)	TRUE (1)	TRUE	TRUE	FALSE
TRUE (1)	UNKNOWN (0.5)	UNKNOWN	TRUE (Short-circuits to TRUE!)	FALSE
FALSE (0)	UNKNOWN (0.5)	FALSE (Short-circuits to FALSE!)	UNKNOWN	TRUE
UNKNOWN (0.5)	UNKNOWN (0.5)	UNKNOWN	UNKNOWN	UNKNOWN (NOT UNKNOWN is UNKNOWN!)

5.5.2 Worked Example 5.4: Tracing 3VL Queries & The Golden WHERE vs. CHECK Asymmetry

GOLDEN EXAM RULE | How WHERE / HAVING vs. CHECK Constraints Treat UNKNOWN

• In a SELECT ... WHERE (or HAVING / JOIN ON) clause: A row is returned ONLY if the predicate evaluates to strictly TRUE! Rows evaluating to FALSE or UNKNOWN are rejected.
• In a CREATE TABLE ... CHECK (...) constraint: A row is accepted unless the predicate evaluates to strictly FALSE! Therefore, a row evaluating to TRUE or UNKNOWN (due to NULL) PASSES a CHECK constraint!

WORKED EXAMPLE 5.4 | Tracing 4 Tricky 3VL Queries on Student (Kabir has DeptID = NULL, CGPA = 6.80)

1. Query: SELECT Name FROM Student WHERE DeptID = 10 OR DeptID <> 10;
• For Kabir (DeptID = NULL): (NULL = 10) is UNKNOWN, and (NULL <> 10) is UNKNOWN. UNKNOWN OR UNKNOWN = UNKNOWN!
• Since WHERE requires strictly TRUE, Kabir is EXCLUDED! (The Law of the Excluded Middle $P \vee \neg P$ fails in 3VL!).

2. Query: SELECT Name FROM Student WHERE DeptID = 10 OR CGPA < 7.00;
• For Kabir: (NULL = 10) is UNKNOWN, (6.80 < 7.00) is TRUE. UNKNOWN OR TRUE = max(0.5, 1) = TRUE! Kabir IS returned!

3. Query: SELECT Name FROM Student WHERE DeptID NOT IN (10, 20, NULL);
• Expands to: DeptID <> 10 AND DeptID <> 20 AND DeptID <> NULL.
• For Asha (DeptID = 10): FALSE AND TRUE AND UNKNOWN = min(0, 1, 0.5) = FALSE.
• For a student in Dept 30: TRUE AND TRUE AND UNKNOWN = min(1, 1, 0.5) = UNKNOWN!
• Result: Returns ZERO rows for the entire table! Never use NOT IN with a nullable list—use NOT EXISTS!

4. CHECK Constraint: CHECK (CGPA >= 5.00) when inserting a student with CGPA = NULL:
• (NULL >= 5.00) evaluates to UNKNOWN. Because CHECK rejects only FALSE, the insert SUCCEEDS unless NOT NULL is also specified!

5.6 Chapter 5 Summary & One-Page Rapid-Revision Cheat Sheet

Chapter 5 Rapid-Revision Formula & Rule Sheet

Concept / Rule	Formula / Key Exam Takeaway
Degree vs. Cardinality	Degree (Arity, n) = number of columns (schema property). Cardinality (m) = number of rows (instance property).
Super Key vs. Candidate Key	Super Key = any unique identifier subset. Candidate Key = MINIMAL Super Key (no proper subset is a Super Key).
Prime vs. Non-Prime Attribute	Prime = belongs to AT LEAST ONE Candidate Key. Non-Prime = belongs to NO Candidate Key.
Super Key Count (1 CK of size k)	2^(n − k) Super Keys.
Super Key Count (2 CKs of size p, q)	2^(n − p) + 2^(n − q) − 2^(n − |K1 ∪ K2|) Super Keys.
Max Candidate Keys (Sperner)	C(n, floor(n / 2)) maximum possible Candidate Keys on n attributes.
Entity Integrity Rule	No attribute of a PRIMARY KEY can ever be NULL (not even one column of a composite PK).
Referential Integrity Rule	Every Foreign Key value must either match an existing parent PK value OR be NULL (if nullable).
3VL Truth Formulas	Set F = 0, U = 0.5, T = 1. Then P AND Q = min(P, Q); P OR Q = max(P, Q); NOT P = 1 − P.
WHERE vs. CHECK with UNKNOWN	WHERE keeps only TRUE (rejects UNKNOWN). CHECK rejects only FALSE (accepts UNKNOWN/NULL!).

Student Revision & Lecture Notes — Chapter 5

Use this page to practice Inclusion–Exclusion Super Key counting, the DML Referential Integrity violation matrix, and 3VL truth table evaluations.

Active Recall Practice Grid — Chapter 5

Prompt	Your Recall Solution
Count Super Keys of R(A,B,C,D,E) with CKs {A} and {B,C}	2^(5−1) + 2^(5−2) − 2^(5−3) = 16 + 8 − 4 = 20 Super Keys.
Which DML operations on Parent & Child can violate FK?	Parent: DELETE or UPDATE of PK. Child: INSERT or UPDATE of FK.
What does WHERE Col = NULL return?	Always 0 rows (UNKNOWN)! Always write WHERE Col IS NULL.
What does NOT IN (1, 2, NULL) return?	Always 0 rows! Because X <> 1 AND X <> 2 AND UNKNOWN is never TRUE.

5.7 Chapter 5 Exam, GATE & Placement Question Bank

Part A: 10 Multiple-Choice Questions (MCQs)

1. Q1. A relation R has 8 attributes and 250 tuples. What are the Degree (Arity) and Cardinality of R, respectively?
   (A) 250 and 8 (B) 8 and 250 (C) 8 and 2^8 (D) 250 and 2000
2. Q2. Which of the following statements about Keys in the Relational Model is ALWAYS TRUE?
   (A) Every Super Key is a Candidate Key (B) Every Candidate Key is a Primary Key (C) Every Candidate Key is a minimal Super Key (D) A relation can have at most one Candidate Key
3. Q3. An attribute that is part of an Alternate Key (a Candidate Key not chosen as the Primary Key) is classified as a:
   (A) Non-Prime Attribute (B) Prime Attribute (C) Foreign Attribute (D) Derived Attribute
4. Q4. Which integrity constraint states that no attribute participating in the Primary Key of a base relation may accept a NULL value?
   (A) Domain Integrity (B) Referential Integrity (C) Entity Integrity (D) Semantic Trigger Integrity
5. Q5. Which of the following statements about a Foreign Key in a relational table is FALSE?
   (A) A Foreign Key may contain duplicate values across rows (B) A Foreign Key may contain NULL values if not declared NOT NULL (C) A Foreign Key can reference the Primary Key of the same table (D) A Foreign Key must always be the Primary Key of the child table

Part A: Multiple-Choice Questions (Continued — Q6 to Q10)

- 6. Q6. Which pair of DML operations can NEVER cause a Referential Integrity violation?
  (A) INSERT into Child; DELETE from Parent (B) INSERT into Parent; DELETE from Child (C) UPDATE PK in Parent; UPDATE FK in Child (D) DELETE from Parent; INSERT into Child
- 7. Q7. In SQL Three-Valued Logic (3VL), what is the truth value of the expression: (TRUE AND UNKNOWN) OR (NOT UNKNOWN)?
  (A) TRUE (B) FALSE (C) UNKNOWN (D) Syntax Error
- 8. Q8. Suppose table T(X INT CHECK (X > 100)) has NO NOT NULL constraint on X. What happens when we run INSERT INTO T VALUES (NULL);?
  (A) Rejected because NULL > 100 is FALSE (B) Accepted because NULL > 100 is UNKNOWN, and CHECK only rejects FALSE (C) Rejected by Entity Integrity (D) X is automatically converted to 101
- 9. Q9. If a parent row in Department is deleted and the child table Student defines DeptID INT REFERENCES Department(DeptID) ON DELETE SET NULL, what happens to the matching students?
  (A) The DELETE is aborted (B) The matching Student rows are deleted (C) The matching Student rows remain, with their DeptID set to NULL (D) The Student table is dropped
- 10. Q10. A relation R(A, B, C, D) has a single Candidate Key {A, B}. How many Super Keys does R have?
  (A) 2 (B) 4 (C) 8 (D) 16

Part B: 5 Short-Answer Questions (2–3 Marks Semester Exam Style)

1. B1. Define Degree (Arity) and Cardinality of a relation. If 50 new rows are inserted and 1 new column is added via ALTER TABLE, how do Degree and Cardinality change?
2. B2. Differentiate clearly between a Super Key, a Candidate Key, a Primary Key, and an Alternate Key with an example.
3. B3. Explain the Entity Integrity Rule and why even a single attribute of a Composite Primary Key cannot be NULL.
4. B4. Compare the five SQL Referential Triggered Actions: RESTRICT, NO ACTION, CASCADE, SET NULL, and SET DEFAULT.
5. B5. Why does SELECT * FROM Student WHERE City = "Patna" OR City <> "Patna" fail to return students whose City is NULL?

Part C: 3 Long-Answer / Essay Questions (5–10 Marks Semester Exam Style)

1. C1. Explain the four mathematical properties of a Relation and compare formal Relational Set Semantics with commercial SQL Bag (Multiset) Semantics.
2. C2. Discuss Domain Integrity, Entity Integrity, and Referential Integrity in detail. Construct the complete 2×3 DML Violation Matrix (INSERT, DELETE, UPDATE on Parent vs. Child) and explain how the DBMS handles each case.
3. C3. Explain Kleene's Three-Valued Logic (3VL) in SQL. Present the truth tables for AND, OR, and NOT, prove why NOT IN with a NULL returns an empty set, and contrast how WHERE and CHECK handle UNKNOWN.

Part D: 4 Combinatorial & Trace Problems

1. D1. (Super-Key Counting Suite): Find the exact number of Super Keys for a relation R(A, B, C, D, E, F, G) with n = 7 attributes under each scenario:
   (a) Only one Candidate Key {A, B, C}.
   (b) Two Candidate Keys {A, B} and {C, D, E}.
   (c) Two overlapping Candidate Keys {A, B, C} and {B, C, D}.
   (d) Three simple Candidate Keys {A}, {B}, and {C}.
2. D2. (Cascading Self-Referencing Delete Trace): Table Emp(EmpID PK, MgrID FK REFERENCES Emp(EmpID) ON DELETE CASCADE) contains 5 rows: (1, NULL), (2, 1), (3, 1), (4, 2), (5, 4). Which rows remain in Emp after executing DELETE FROM Emp WHERE EmpID = 2;?
3. D3. (3VL Expression Evaluation): Given A = 10, B = 20, C = NULL, evaluate the exact 3VL truth value (TRUE, FALSE, or UNKNOWN) of:
   (i) (A < B) AND (C = 10) (ii) (A < B) OR (C = 10) (iii) NOT (A > B OR C IS NULL) (iv) (C = C)
4. D4. (Deduction from Instance): Given a 4-row relation instance R(A, B, C) with rows (1, 10, 5), (2, 10, 6), (3, 20, 5), (4, 30, 7), list which single attributes CANNOT be a Candidate Key and explain why.

Part E: 5 GATE CS Style Questions (MCQ, MSQ, and NAT)

1. E1. [GATE NAT — 2 Marks] A relation R(A1, A2, A3, A4, A5, A6) has 6 attributes. If {A1, A2} and {A2, A3} are the only two Candidate Keys of R, the total number of Super Keys of R is _____.
2. E2. [GATE NAT — 2 Marks] Consider three tables R1(_A_, B), R2(_C_, A, D), and R3(_E_, C) with the following foreign keys:
   • In R2: A REFERENCES R1(A) ON DELETE CASCADE
   • In R3: C REFERENCES R2(C) ON DELETE SET NULL
   Suppose R1 contains {(1,10), (2,20)}; R2 contains {(100,1,5), (101,1,6), (102,2,7)}; and R3 contains {(500,100), (501,101), (502,102)}. When DELETE FROM R1 WHERE A = 1; is executed, the number of rows deleted from R2 is _____, and the number of rows deleted from R3 is _____.
3. E3. [GATE MSQ — 2 Marks] Let R(A, B, C, D, E) have Candidate Keys {A, B} and {C}. Which of the following statements are TRUE?
   (A) A, B, and C are Prime Attributes, while D and E are Non-Prime Attributes.
   (B) {A, C} is a Super Key of R, but NOT a Candidate Key.
   (C) The total number of Super Keys of R is 20.
   (D) Attribute C can accept NULL values if {A, B} is chosen as the Primary Key and C is not declared NOT NULL.
4. E4. [GATE NAT — 2 Marks] Using Sperner's Theorem, the maximum number of Candidate Keys that any relation schema R with n = 5 attributes can possibly have is _____.
5. E5. [GATE MCQ — 2 Marks] Table T(id INT PRIMARY KEY, val INT) contains 3 rows: (1, 10), (2, 20), (3, NULL). How many rows are returned by the query:
   SELECT id FROM T WHERE val NOT IN (SELECT val FROM T WHERE id >= 2);
   (A) 3 (B) 2 (C) 1 (D) 0

Part F: 5 Viva / Technical Placement Interview Questions

1. F1. "In PostgreSQL or MySQL, if I create a column Email VARCHAR(100) UNIQUE (without NOT NULL), how many rows can have Email = NULL?"
2. F2. "When should you use a Surrogate Primary Key (like BIGINT GENERATED ALWAYS AS IDENTITY or UUIDv7) vs. a Natural Business Key?"
3. F3. "Why is UUIDv4 bad for B+ Tree Primary Key performance compared to auto-increment BIGINT or time-ordered UUIDv7?"
4. F4. "What is the exact difference between ON DELETE RESTRICT and ON DELETE NO ACTION in SQL?"
5. F5. "How do you safely rewrite a SQL query that uses NOT IN (subquery) so it never silently returns 0 rows when the subquery contains a NULL?"

Part G: 3 Hands-On SQL Labs & Part H: 3 "Spot the Mistake" Challenges

1. Lab 5.1: Write SQL to test ON DELETE SET NULL vs. ON DELETE CASCADE between Department and Student.
2. Lab 5.2: Write a query using IS DISTINCT FROM (or COALESCE) to safely compare nullable columns where NULL = NULL should be treated as true.
3. Lab 5.3: Demonstrate how CHECK (Bonus <= Salary) behaves when Bonus is NULL vs. when Salary is NULL.
4. Mistake 5.1: A developer writes WHERE ManagerID = NULL to find top-level executives.
5. Mistake 5.2: A student claims that because {StudentID} is a Candidate Key, {StudentID, Name} is also a Candidate Key.
6. Mistake 5.3: A student claims that deleting a child row from Enrollment can cause a Foreign Key violation in Student.

5.8 Chapter 5 Complete Answer Key & Worked Solutions

Solutions to Part A: Multiple-Choice Questions (Q1 to Q10)

Answer Key & Detailed Explanations for Part A MCQs

Q#	Ans	Detailed Step-by-Step Explanation
Q1	(B)	Degree (Arity) = number of attributes (8); Cardinality = number of tuples (250).
Q2	(C)	By definition, a Candidate Key is a minimal Super Key (no proper subset is a Super Key).
Q3	(B)	Any attribute belonging to ANY Candidate Key (including Alternate Keys) is a Prime Attribute.
Q4	(C)	Entity Integrity states that no attribute of a Primary Key can be NULL.
Q5	(D)	A Foreign Key does NOT need to be the Primary Key of the child table (e.g., DeptID in Student is a non-key attribute).
Q6	(B)	Inserting a new Parent row and Deleting an existing Child row can NEVER leave orphan Foreign Keys!
Q7	(C)	(TRUE AND UNKNOWN) = UNKNOWN; (NOT UNKNOWN) = UNKNOWN; UNKNOWN OR UNKNOWN = UNKNOWN.
Q8	(B)	In SQL, CHECK constraints only reject rows where the predicate evaluates to strictly FALSE. UNKNOWN passes!
Q9	(C)	ON DELETE SET NULL retains the child Student rows and sets their DeptID foreign key to NULL.
Q10	(B)	With n = 4 and 1 CK of size k = 2, number of Super Keys = 2^(4 − 2) = 2^2 = 4 ({AB}, {ABC}, {ABD}, {ABCD}).

Solutions to Part B: Short-Answer Questions (B1 to B5)

- Answer B1 (Degree vs. Cardinality): Degree is the number of columns in the schema; Cardinality is the number of rows in the current instance. Inserting 50 rows increases Cardinality by 50 (Degree unchanged); adding 1 column via ALTER TABLE increases Degree by 1 (Cardinality unchanged).
- Answer B2 (SK, CK, PK, Alternate Key): In Student(StudentID, RollNo, Name, City): {StudentID, Name} is a Super Key; {StudentID} and {RollNo} are minimal Super Keys (Candidate Keys); if {StudentID} is chosen as the Primary Key, then {RollNo} becomes an Alternate Key.
- Answer B3 (Entity Integrity & Composite PKs): Entity Integrity forbids NULL in any Primary Key attribute because the PK serves as the unique tuple address. If (StudentID, SectionID) allowed (101, NULL), we could not determine which section student 101 enrolled in or whether two (101, NULL) rows represent the same enrollment.
- Answer B4 (5 Referential Actions): RESTRICT rejects parent delete/update immediately if children exist; NO ACTION rejects at the end of the statement/deferred check; CASCADE propagates the delete/update to children; SET NULL sets child FKs to NULL; SET DEFAULT sets child FKs to their default parent key.
- Answer B5 (Why City = "Patna" OR City <> "Patna" Misses NULLs): When City is NULL, both comparisons evaluate to UNKNOWN, and UNKNOWN OR UNKNOWN = UNKNOWN. Since WHERE filters out UNKNOWN, rows with NULL City are omitted (unless OR City IS NULL is added).

Solutions to Part C (C1–C3) & Part D: Combinatorial & Trace Problems (D1 to D4)

Worked Solutions D1 | Super-Key Counting Suite on n = 7 Attributes

(a) Single CK {A,B,C} (k = 3): |SK| = 2^(7−3) = 2^4 = 16 Super Keys.
(b) Disjoint CKs {A,B} (p = 2) and {C,D,E} (q = 3), union size u = 5:
|SK| = 2^(7−2) + 2^(7−3) − 2^(7−5) = 32 + 16 − 4 = 44 Super Keys.
(c) Overlapping CKs {A,B,C} (p = 3) and {B,C,D} (q = 3), union {A,B,C,D} has size u = 4:
|SK| = 2^(7−3) + 2^(7−3) − 2^(7−4) = 16 + 16 − 8 = 24 Super Keys.
(d) Three simple CKs {A}, {B}, {C} (m = 3): |SK| = 2^7 − 2^(7−3) = 128 − 16 = 112 Super Keys.

Worked Solutions D2, D3, and D4

Solution D2 (Self-Referencing Cascade Trace):
• Deleting EmpID = 2 cascades to its direct child (4, 2) [EmpID = 4], which in turn cascades to EmpID = 4's child (5, 4) [EmpID = 5]!
• Surviving rows in Emp: (1, NULL) and (3, 1) (2 rows remain; 3 rows deleted).

Solution D3 (3VL Expressions with A=10, B=20, C=NULL):
• (i) (10 < 20) AND (NULL = 10) -> TRUE AND UNKNOWN = UNKNOWN.
• (ii) (10 < 20) OR (NULL = 10) -> TRUE OR UNKNOWN = TRUE.
• (iii) NOT (10 > 20 OR NULL IS NULL) -> NOT (FALSE OR TRUE) = NOT (TRUE) = FALSE.
• (iv) (NULL = NULL) -> UNKNOWN!

Solution D4 (Deduction from Instance): Attribute B has duplicates (10 appears twice) and Attribute C has duplicates (5 appears twice). Therefore, neither {B} nor {C} can be a Candidate Key! ({A} has distinct values 1,2,3,4 so it is consistent with being a key, though schema FDs are needed to prove it).

Solutions to Part E: GATE CS Questions (E1 to E5)

Step-by-Step GATE CS Solutions for Questions E1 to E5

GATE Q#	Correct Answer	Complete Step-by-Step Derivation
E1 (NAT)	24	n = 6, K1 = {A1,A2} (p=2), K2 = {A2,A3} (q=2), K1 ∪ K2 = {A1,A2,A3} (u=3). |SK| = 2^(6−2) + 2^(6−2) − 2^(6−3) = 16 + 16 − 8 = 24.
E2 (NAT)	2 from R2; 0 from R3	Deleting A=1 from R1 cascades to R2, deleting rows C=100 and C=101 (2 rows deleted from R2). Because R3 has ON DELETE SET NULL, rows 500 and 501 in R3 are NOT deleted—their C column is simply set to NULL! So 0 rows are deleted from R3.
E3 (MSQ)	(A), (B), (C), (D)	All four statements are TRUE! |SK| = 2^(5−2) + 2^(5−1) − 2^(5−3) = 8 + 16 − 4 = 20. And an Alternate Key (UNIQUE) allows NULLs unless declared NOT NULL.
E4 (NAT)	10	By Sperner's Theorem, for n = 5, max Candidate Keys = C(5, floor(5/2)) = C(5, 2) = (5 × 4)/2 = 10.
E5 (MCQ)	(D) 0 rows	The subquery returns (20, NULL). For id=1 (val=10): 10 NOT IN (20, NULL) <=> (10 <> 20) AND (10 <> NULL) <=> TRUE AND UNKNOWN = UNKNOWN! Thus 0 rows are returned.

Solutions to Part F: Viva & Technical Placement Interview Questions (F1 to F5)

- Answer F1 (Multiple NULLs in a UNIQUE Column): In standard SQL and PostgreSQL/MySQL, a UNIQUE constraint allows MULTIPLE rows to contain NULL! Why? Because in 3VL, NULL = NULL is UNKNOWN (not TRUE), so two NULLs are not considered equal duplicates! (Note: PostgreSQL 15+ added UNIQUE NULLS NOT DISTINCT if you want at most one NULL).
- Answer F2 (Surrogate Key vs. Natural Key): Natural keys (like Email or Phone) can change over a user's lifetime, forcing expensive ON UPDATE CASCADE updates across all child tables. A compact immutable Surrogate PK (BIGINT IDENTITY) keeps Foreign Keys small (8 bytes) and immune to business changes—while you still place a UNIQUE constraint on the Natural Key!
- Answer F3 (Why Random UUIDv4 Hurts B+ Tree Inserts): UUIDv4 values are completely random across the 128-bit space, so every new INSERT lands in a random leaf page of the B+ Tree—causing frequent page splits, buffer pool churn, and random disk I/O. Sequential BIGINT or timestamp-prefixed UUIDv7 always appends to the rightmost B+ Tree leaf page!
- Answer F4 (RESTRICT vs. NO ACTION): Both reject deleting/updating a parent row that has children. However, RESTRICT checks immediately before the row change, whereas NO ACTION can be declared DEFERRABLE INITIALLY DEFERRED so constraint checking waits until COMMIT.
- Answer F5 (Fixing NOT IN): Replace WHERE x NOT IN (SELECT y FROM S) with either WHERE NOT EXISTS (SELECT 1 FROM S WHERE S.y = x) or filter out NULLs explicitly inside the subquery: WHERE x NOT IN (SELECT y FROM S WHERE y IS NOT NULL).

Solutions to Part G (Labs 5.1–5.3) & Part H ("Spot the Mistake" 5.1–5.3)

SQL Solutions for Labs 5.1, 5.2, and 5.3

```
-- Lab 5.2: Null-Safe Equality Comparison Using IS NOT DISTINCT FROM
SELECT s1.StudentID, s2.StudentID
FROM Student s1
JOIN Student s2 ON s1.StudentID < s2.StudentID
WHERE s1.DeptID IS NOT DISTINCT FROM s2.DeptID;
-- Treats (NULL, NULL) as TRUE (both students have undeclared department)!

-- Safe NOT EXISTS replacement for NOT IN:
SELECT Name FROM Student s
WHERE NOT EXISTS (
    SELECT 1 FROM Department d WHERE d.DeptID = s.DeptID
);
```

Diagnostic Solutions 5.1, 5.2, and 5.3

• Mistake 5.1 (WHERE ManagerID = NULL): Always evaluates to UNKNOWN and returns 0 rows! Must write WHERE ManagerID IS NULL.
• Mistake 5.2 (Confusing Super Key with Candidate Key): Because {StudentID} alone is already a Super Key, {StudentID, Name} is a non-minimal Super Key, NOT a Candidate Key!
• Mistake 5.3 (Deleting a Child Row Never Violates FK): Deleting from the referencing (child) table Enrollment only removes a reference; it can never create an orphan in the parent Student table.

END OF CHAPTER 5 | Up Next: Chapter 6 — Relational Algebra and Relational Calculus

In Chapter 6, we master the formal query languages behind SQL: Procedural Relational Algebra (σ, π, ∪, −, ×, ρ, Joins, and Division ÷), GATE Cardinality Bounds, and Declarative Tuple/Domain Relational Calculus!